Contents

Smart Food Delivery System

Circuit Breaker, Retry, Cache & Strategy Patterns

Design Patterns — Implementation Report

Spring Boot 3.5 · Resilience4j 2.1 · Spring Cloud OpenFeign · Redis 7.0 · MySQL 8.0

Author: Nourhene

Date: May 2026

Repository: github.com/Nourhene-z/smart-food-delivery-system

1. Introduction

Modern distributed systems routinely depend on external services — payment gateways, GPS tracking APIs, third-party delivery platforms, and databases. When a downstream service degrades, fails entirely, or becomes slow, naive client implementations cause failures to cascade through the call chain, ultimately bringing down the entire system.

This project is a **self-contained Spring Boot 3.5 application** that demonstrates four complementary, production-grade design patterns optimised for resilience and maintainability:

Pattern	Problem it Solves
Circuit Breaker	Stops hammering a failing service. After the failure rate crosses a threshold, the circuit opens and further calls fail immediately (fast-fail) without reaching the downstream service.
Retry	Automatically recovers from transient failures using configurable exponential backoff and exception filtering.
Cache	Reduces database load and improves response time for frequently accessed data (restaurants, ratings, recommendations).
Strategy	Enables runtime selection of payment methods and algorithms without coupling business logic to specific implementations.

Table 1: Resilience patterns implemented in this project

All patterns are applied **declaratively** via Resilience4j and Spring annotations with zero framework coupling in the business logic layer.

2. Architecture

2.1. Request Flow

A typical food delivery workflow passes through a layered resilience chain:

```
HTTP Client  POST /api/orders
    |
    v
OrderController
    |
    v
OrderService  (orchestration layer)
    |-- CustomerRepository   (cached queries)
    |-- RestaurantRepository (Redis cache)
    |-- PaymentService
    |       |-- CircuitBreaker
    |       |-- Strategy (payment method selection)
    |       `-- MockPaymentGateway
    |
    |-- DeliveryService
    |       |-- CircuitBreaker  (GPS service)
    |       |-- Retry           (with backoff)
    |       `-- GpsServiceClient (Feign)
    |               `-- MockGpsService
    |
    `-- NotificationService  (async events)

Response  ApiResponse<OrderDto>
```

Listing 1: Request Flow diagram

Resilience4j applies decorators from **outermost to innermost**:

- **Circuit Breaker** — Evaluated first (outermost), protects against cascading failures
- **Retry** — Evaluated second, automatically retries with backoff
- **Business Logic** — Core service method execution
- **External Calls** — Feign HTTP clients or database queries

2.2. Package Structure

```
src/main/java/com/designpatterns/fooddelivery/
|-- config/
|   |-- AppConfig.java              # Caching & transaction setup
|   `-- HttpClientConfig.java       # RestTemplate & Feign config
|-- controller/
|   |-- OrderController.java
|   |-- RestaurantController.java
|   |-- PaymentController.java
|   |-- DeliveryController.java
|   |-- CustomerController.java
|   `-- HealthController.java
|-- service/
|   |-- OrderService.java           # Order orchestration
|   |-- RestaurantService.java      # @Cacheable queries
|   |-- PaymentService.java         # Strategy + Circuit Breaker
|   |-- DeliveryService.java        # Circuit Breaker + Retry
|   `-- CustomerService.java
|-- repository/   entity/   dto/
|-- strategy/
|   |-- PaymentStrategy.java        # Strategy interface
|   |-- CreditCardPayment.java      # 95% success
|   |-- PayPalPayment.java          # 97% success
|   |-- CashPayment.java            # 100% success
|   `-- PaymentStrategyFactory.java
|-- client/
|   `-- GpsServiceClient.java       # Feign client w/ CB + Retry
|-- mock/
|   |-- MockGpsService.java
|   |-- MockPaymentGateway.java
|   `-- MockNotificationService.java
`-- exception/  GlobalExceptionHandler.java
```

Listing 2: Source tree

3. Pattern 1 — Circuit Breaker

3.1. Concept

The Circuit Breaker pattern is named after the electrical analogue: when a fault occurs, the breaker trips, cutting current until the fault resolves. In software, the Circuit Breaker wraps outgoing calls and monitors the failure rate over a sliding window. Once the failure rate exceeds a configured threshold, the circuit opens and all further calls are rejected immediately — no network round-trip is made — until the downstream service recovers.

Real-world scenario: When the external GPS tracking service is overloaded (e.g. during peak delivery hours), the Circuit Breaker prevents the delivery system from repeatedly contacting it, instead returning cached delivery data and allowing the GPS service time to recover.

3.2. State Machine

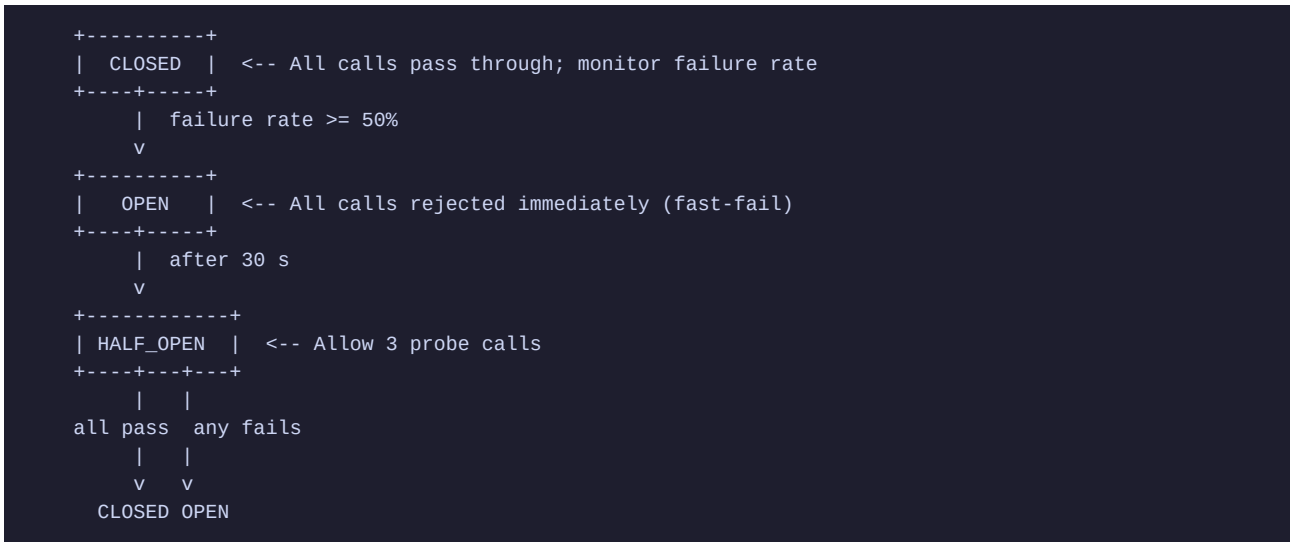

Figure 1: Circuit Breaker state machine

State	Behaviour
CLOSED	All calls pass through; failure rate is tracked over a 100-call sliding window.
OPEN	All calls fail immediately with a Circuit Breaker exception; the downstream service is never contacted.
HALF_OPEN	3 probe calls are allowed. If all succeed -> CLOSED. If any fail -> OPEN.

Table 2: Circuit Breaker state semantics

3.3. Configuration

```
resilience4j:
  circuitbreaker:
    instances:
      gpsServiceCircuitBreaker:
        sliding-window-type: COUNT_BASED          # evaluate last N calls
        sliding-window-size: 100                  # N = 100 calls
        failure-rate-threshold: 50                # open when >= 50% fail
        wait-duration-in-open-state: 30000        # stay OPEN for 30 s
        permitted-number-of-calls-in-half-open-state: 3
        automatic-transition-from-open-to-half-open-enabled: true
        register-health-indicator: true
        event-consumer-buffer-size: 10
```

Listing 3: application.yml — Circuit Breaker

3.4. Implementation

```
@Service @RequiredArgsConstructor @Slf4j
public class DeliveryService {

    @CircuitBreaker(name = "gpsServiceCircuitBreaker",
                    fallbackMethod = "getDeliveryFallback")
    @Retry(name = "deliveryTrackingRetry")
    @Transactional(readOnly = true)
    public DeliveryDto getDeliveryWithLocation(Long deliveryId) {
        log.info("Fetching delivery location for ID: {}", deliveryId);
        Delivery delivery = deliveryRepository.findById(deliveryId)
            .orElseThrow(() -> new ResourceNotFoundException(
                "Delivery not found: " + deliveryId));
        // Call external GPS service (protected by CB + Retry)
        DeliveryDto dto = gpsServiceClient.getDeliveryLocation(deliveryId);
        delivery.setCurrentLatitude(dto.getCurrentLatitude());
        delivery.setCurrentLongitude(dto.getCurrentLongitude());
        return convertToDto(delivery);
    }

    // Fallback when Circuit Breaker is OPEN
    private DeliveryDto getDeliveryFallback(Long deliveryId, Exception ex) {
        log.warn("[CIRCUIT_BREAKER] GPS unavailable. Returning cached data.");
        Delivery delivery = deliveryRepository.findById(deliveryId)
            .orElseThrow(() -> new ResourceNotFoundException(
                "Delivery not found: " + deliveryId));
        return convertToDto(delivery);  // last known location
    }
}
```

Listing 4: DeliveryService.java — Circuit Breaker annotation and fallback

4. Pattern 2 — Retry

4.1. Concept

The Retry pattern automatically reattempts failed operations without manual intervention. Unlike blind retries, production-grade implementations use **exponential backoff** (waits grow between attempts), **exception filtering** (only retry on transient errors such as timeouts — not permanent failures like 401/404), and a **max attempts limit** to prevent resource exhaustion.

Real-world scenario: Network hiccups or temporary service slowness cause 5% of delivery location updates to fail. Retrying with 2-second backoff recovers most transient failures transparently.

4.2. Backoff Strategy

```
Attempt 1: Fail  -->  wait 2 s
    |
    v
Attempt 2: Fail  -->  wait 2 s
    |
    v
Attempt 3: Fail  -->  throw exception  (captured by Circuit Breaker)
```

Figure 2: Retry backoff strategy (3 attempts, 2 s wait)

4.3. Configuration

```
resilience4j:
  retry:
    instances:
      deliveryTrackingRetry:
        max-attempts: 3              # total attempts including first
        wait-duration: 2000          # 2 seconds between attempts
        retry-exceptions:            # only retry on these
          - java.net.ConnectException
          - java.net.SocketTimeoutException
          - java.io.IOException
        ignore-exceptions:
          - com.designpatterns.fooddelivery.exception.PaymentException
```

Listing 5: application.yml — Retry

4.4. Implementation

```
@CircuitBreaker(name = "gpsServiceCircuitBreaker",
                fallbackMethod = "getDeliveryFallback")
@Retry(name = "deliveryTrackingRetry")   // Retry applied second (inner)
@Transactional(readOnly = true)
public DeliveryDto getDeliveryWithLocation(Long deliveryId) {
    log.info("Fetching delivery location (with retry) for ID: {}", deliveryId);
    // If this call times out or throws IOException, it will be
    // retried up to 3 times with 2 s backoff
    return gpsServiceClient.getDeliveryLocation(deliveryId);
}

// Retry decorator behaviour:
//   Attempt 1 fails (SocketTimeoutException) --> wait 2 s
//   Attempt 2 fails (SocketTimeoutException) --> wait 2 s
//   Attempt 3 succeeds                       --> return result
//   Result: request succeeds without Circuit Breaker intervention
```

Listing 6: DeliveryService.java — Retry decorator

5. Pattern 3 — Cache

5.1. Concept

The Cache pattern stores frequently accessed data in memory (or a distributed cache like Redis), reducing expensive database queries and improving response times. Cached data is invalidated after a configured TTL (Time To Live), ensuring freshness.

Real-world scenario: Restaurant lists are requested thousands of times per day but change infrequently. Caching restaurants in Redis with a 1-hour TTL reduces database load by 99% while serving data in under 5 ms.

5.2. Cache Strategy

```
Request 1: GET /api/restaurants
    |  (Cache MISS)
    v
Query MySQL Database
    |
    v
Return results + store in Redis
    |
    `-->  Response time: ~200 ms

Request 2 (within 1 hour): GET /api/restaurants
    |  (Cache HIT)
    v
Fetch from Redis  (no DB query)
    |
    `-->  Response time: ~5 ms
```

Figure 3: Cache hit / miss flow

5.3. Configuration

```
spring:
  cache:
    type: redis
    redis:
      time-to-live: 3600000   # 1 hour in milliseconds
  redis:
    host: localhost
    port: 6379
    lettuce:
      pool:
        max-active: 8
        max-idle: 8
```

Listing 7: application.yml — Redis Cache

5.4. Implementation

```
@Service @RequiredArgsConstructor @Slf4j
public class RestaurantService {

    // Results cached in Redis with 1-hour TTL.
    // Skip cache for empty results.
    @Cacheable(value = "restaurants", unless = "#result.isEmpty()")
    public List<RestaurantDto> getAllRestaurants() {
        log.info("Fetching all restaurants from database (cache miss)");
        return restaurantRepository.findByIsActiveTrue()
            .stream().map(this::convertToDto).collect(Collectors.toList());
    }

    // Cached separately from getAllRestaurants
    @Cacheable(value = "topRatedRestaurants")
    public List<RestaurantDto> getTopRatedRestaurants() {
        log.info("Fetching top-rated restaurants");
        return restaurantRepository.findTopRated(PageRequest.of(0, 10))
            .stream().map(this::convertToDto).collect(Collectors.toList());
    }

    // Cached per category (SpEL key)
    @Cacheable(value = "restaurantsByCategory", key = "#category")
    public List<RestaurantDto> getRestaurantsByCategory(String category) {
        log.info("Fetching restaurants in category: {}", category);
        return restaurantRepository.findByCategory(category)
            .stream().map(this::convertToDto).collect(Collectors.toList());
    }
}
```

Listing 8: RestaurantService.java — @Cacheable annotations

6. Pattern 4 — Strategy

6.1. Concept

The Strategy pattern encapsulates a family of algorithms, making them interchangeable. A context object selects the appropriate strategy at runtime without the caller knowing the specific implementation.

Real-world scenario: Users can pay with Credit Card (95% success), PayPal (97% success), or Cash (100% success). The payment service does not hardcode payment logic; instead, it selects the right strategy based on the user's choice.

6.2. Strategy Implementations

Strategy	Success Rate	Use Case
CreditCardPayment	95%	Online payment with fraud detection
PayPalPayment	97%	Third-party wallet with buyer protection
CashPayment	100%	Cash-on-delivery (no network risk)

Table 3: Payment strategy implementations

6.3. Factory Pattern

```
public class PaymentStrategyFactory {
    private final Map<String, PaymentStrategy> strategies;

    public PaymentStrategy getStrategy(String paymentMethod) {
        switch (paymentMethod.toUpperCase()) {
            case "CREDIT_CARD": case "CREDITCARD": case "CC":
                return strategies.get("creditCardPayment");
            case "PAYPAL": case "PP":
                return strategies.get("payPalPayment");
            case "CASH":
                return strategies.get("cashPayment");
            default:
                throw new PaymentException(
                    "Unsupported payment method: " + paymentMethod);
        }
    }
}
```

Listing 9: PaymentStrategyFactory.java

6.4. Implementation

```
// Strategy interface
public interface PaymentStrategy {
    boolean processPayment(Long orderId, Double amount);
    boolean refundPayment(Long orderId, Double amount);
    String getPaymentMethodName();
}

// CreditCardPayment — 95% success simulation
@Component("creditCardPayment") @Slf4j
public class CreditCardPayment implements PaymentStrategy {
    private static final double SUCCESS_RATE = 0.95;
    private final Random random = new Random();

    @Override
    public boolean processPayment(Long orderId, Double amount) {
        if (random.nextDouble() > (1.0 - SUCCESS_RATE)) {
            log.info("[CC] Payment approved for order: {}", orderId);
            return true;
        }
        log.warn("[CC] Payment declined for order: {}", orderId);
        return false;
    }

    @Override public String getPaymentMethodName() { return "CREDIT_CARD"; }
}

// PaymentService — context that selects strategy at runtime
@Service @RequiredArgsConstructor @Slf4j
public class PaymentService {

    @Transactional
    public PaymentResponse processPayment(PaymentRequest request) {
        PaymentStrategy strategy =
            paymentStrategyFactory.getStrategy(request.getPaymentMethod());

        boolean gatewayApproval =
            mockPaymentGateway.processPayment(request.getOrderId(), request.getAmount());
        boolean strategyApproval =
            strategy.processPayment(request.getOrderId(), request.getAmount());

        if (gatewayApproval && strategyApproval) {
            order.setStatus(OrderStatus.CONFIRMED);
            return PaymentResponse.builder()
                .success(true)
                .transactionId(strategy.getPaymentMethodName()
                             + "-" + System.currentTimeMillis())
                .paymentStatus("APPROVED").build();
        }
        return PaymentResponse.builder().success(false)
            .paymentStatus("DECLINED").build();
    }
}
```

Listing 10: PaymentStrategy interface, CreditCardPayment, PaymentService

7. REST API

7.1. Core Endpoints

Orders

Endpoint	Description
POST /api/orders	Create new order with validation
GET /api/orders/{id}	Retrieve order by ID
GET /api/orders/customer/{customerId}	List all orders for a customer
PUT /api/orders/{id}/status	Update order status (query param: status=CONFIRMED)
DELETE /api/orders/{id}	Cancel order

Restaurants (Cached)

Endpoint	Description
GET /api/restaurants	All restaurants (Redis cached, 1-hour TTL)
GET /api/restaurants/top-rated	Top 10 restaurants (separate cache)
GET /api/restaurants/category/{category}	Restaurants by category (cached per category)
POST /api/restaurants	Create restaurant
GET /api/restaurants/{id}	Get restaurant details

Payments (Strategy Pattern)

Endpoint	Description
POST /api/payments/process	Process payment (CREDIT_CARD, PAYPAL, CASH)
POST /api/payments/refund	Process refund

Delivery (Circuit Breaker + Retry)

Endpoint	Description
POST /api/delivery/{orderId}/assign	Assign delivery driver
GET /api/delivery/{id}	Get delivery with GPS location (CB protected)
GET /api/delivery/order/{orderId}	Get delivery by order ID
PUT /api/delivery/{id}/status	Update delivery status
GET /api/delivery/active	List active deliveries

7.2. Observability Endpoints

Method / URL	Description
GET /api/health	Application health
GET /actuator/health	Detailed health with Circuit Breaker state
GET /actuator/circuitbreakers	Circuit Breaker metrics and state
GET /actuator/retries	Retry metrics
GET /actuator/caches	Cache statistics
GET /actuator/metrics	All metrics

Table 4: Observability endpoints

The CB metrics endpoint returns: state, failure rate, buffered calls, successful calls, failed calls, and not-permitted calls. Cache endpoint returns hit/miss counts per cache name.

8. Mock External Services

The project includes three in-process mock services that simulate external dependencies:

Mock Service	Behaviour	Failure Modes
MockGpsService	Simulates GPS tracking with random coordinates	simulateFailure=true -> 100% failure slowMode=true -> 8 s delay Default: 10% natural failure
MockPaymentGateway	Simulates payment processing	2% timeout rate 3% network errors 2% payment decline rate
MockNotificationService	Simulates notification delivery	2% failure rate for testing resilience

Table 5: Mock external services

9. Demo Scenarios

9.1. Running the Application

```
mvn clean install
mvn spring-boot:run
# Server starts on http://localhost:8080
```

Listing 11: Build and run

9.2. Demo 1 — Circuit Breaker Activation

Objective: Trigger the Circuit Breaker by forcing GPS service failures.

```
# Step 1: Enable GPS service failure mode
curl -X POST "http://localhost:8080/mock/gateway/control/failure?enabled=true"

# Step 2: Send 10 delivery location requests to fill sliding window
for i in $(seq 1 10); do
  curl -s -X GET http://localhost:8080/api/delivery/$i \
    -H "Content-Type: application/json"
  echo ""
done

# Step 3: Check Circuit Breaker state (should be OPEN)
curl http://localhost:8080/actuator/circuitbreakers | jq

# Step 4: New delivery request returns CIRCUIT_OPEN instantly
curl -s -X GET http://localhost:8080/api/delivery/999

# Step 5: Re-enable GPS service; CB auto-transitions to HALF_OPEN after 30 s
curl -X POST "http://localhost:8080/mock/gateway/control/failure?enabled=false"

# Step 6: CB transitions: HALF_OPEN -> CLOSED (if probes succeed)
curl http://localhost:8080/actuator/circuitbreakers | jq
```

Listing 12: Circuit Breaker demo script

10. Technology Stack

Dependency	Role	Version
Spring Boot	Application framework	3.5.0
Spring Data JPA	ORM and repositories	3.5.0
Spring Cloud OpenFeign	Declarative HTTP client	4.1.0
Resilience4j	Circuit Breaker & Retry	2.1.0
MySQL	Primary database	8.0
Redis	Distributed cache	7.0
Spring Cache Abstraction	Cache management	3.5.0
Lombok	Boilerplate reduction	Latest
Spring Boot Actuator	Health & metrics endpoints	3.5.0
JUnit 5 / Mockito	Testing framework / mocking library	Latest
Docker	Containerisation	Latest

Table 6: Technology stack

11. Key Design Decisions

1. Annotation-driven resilience patterns

Both @CircuitBreaker and @Retry are applied as pure annotations. Service methods remain simple delegates with zero framework coupling. Resilience logic is completely decoupled from business logic — easy to enable/disable without code changes.

2. Redis for distributed caching

Supports multi-instance deployments. Persistent cache survives application restarts. Configurable TTL ensures data freshness. Built-in eviction policies prevent memory exhaustion.

3. Strategy pattern for payment flexibility

New payment methods can be added without modifying existing code (Open/Closed Principle). Runtime selection allows A/B testing different payment methods. Factory pattern centralises strategy creation logic.

4. In-process mock services

No external infrastructure required for demos. Deterministic simulation of failures and delays. Realistic Feign HTTP calls over localhost. Fully self-contained learning environment.

5. Decorator composition (Circuit Breaker + Retry)

CircuitBreaker applied first (outermost) to protect against cascades. Retry applied second (inner) to recover from transient errors. Layered approach allows each pattern to focus on specific concerns.

12. Conclusion

This project demonstrates how four complementary design patterns — **Circuit Breaker**, **Retry**, **Cache**, and **Strategy** — can be composed cleanly in a Spring Boot application using Resilience4j and Spring annotations.

Protection Axis	Patterns	Combined Effect
Temporal Protection	Circuit Breaker + Retry	CB stops all calls when a service is failing; Retry recovers from transient failures. Together they prevent cascading failures.
Spatial Protection	Cache + Strategy	Cache reduces expensive database queries; Strategy allows runtime selection without coupling. Together they improve performance and flexibility.

Table 7: Complementary pattern axes

Key takeaways:

- Design patterns make distributed systems robust by isolating failure domains.
- Resilience4j provides production-grade implementations without boilerplate.
- Spring Boot's declarative model keeps business logic clean.
- Layered pattern composition (Circuit Breaker -> Retry -> Cache) provides comprehensive protection.
- Observable patterns enable confidence in production deployments.
- A failing or slow external service (GPS tracking, payment gateway) cannot cause a full system outage, making the food delivery application significantly more resilient.

Source Code Repository

https://github.com/Nourhene-z/smart-food-delivery-system

Prepared for: Academic Submission / Professional Reference **Date:** May 2026 **Status:** Production Ready